

Kachelspiel

Manuel und Julian spielen im Altersheim ein Spiel auf einem Gitter aus $2N \times 2M$ quadratischen Zellen. Die Zeilen sind von oben nach unten mit 0 bis $2N - 1$ nummeriert, die Spalten von links nach rechts mit 0 bis $2M - 1$. Für $0 \leq i < 2N$ und $0 \leq j < 2M$ bezeichnen wir die Zelle in Zeile i und Spalte j mit (i, j) .

Manuel gibt Julian $N \cdot M$ **Blöcke**, einen nach dem anderen. Jeder Block ist ein 2×2 -Quadrat, bestehend aus vier 1×1 -**Kacheln**. Manuel hat jede Kachel des Blocks entweder schwarz oder weiß eingefärbt und dabei sichergestellt, dass **mindestens eine Kachel weiß ist**.

Julian muss jeden Block **sofort nach Erhalt** auf dem Spielfeld platzieren, ohne zu wissen, welche Blöcke er später erhalten wird. Die Blöcke dürfen nicht gedreht werden. Jeder Block muss vollständig innerhalb des Gitters platziert werden und bedeckt genau vier Gitterzellen. Außerdem muss die obere linke Kachel jedes Blocks eine Zelle bedecken, deren Zeilen- und Spaltenkoordinaten beide gerade sind. Jede Zelle des Gitters kann von höchstens einem Block bedeckt werden.

Manuel gewinnt das Spiel, wenn nach dem Platzieren eines beliebigen Blocks ein 2×2 -Quadrat existiert, das vollständig mit vier schwarzen Kacheln bedeckt ist. Formal ausgedrückt: Manuel gewinnt, wenn es Zellen (a, b) , $(a + 1, b)$, $(a, b + 1)$, $(a + 1, b + 1)$ mit $0 \leq a < 2N - 1$ und $0 \leq b < 2M - 1$ gibt, die vollständig mit schwarzen Kacheln bedeckt sind. Die Indizes a und b müssen nicht gerade sein.

Julian gewinnt, wenn er alle $N \cdot M$ Blöcke platziert, ohne dass Manuel gewinnt. Beachte, dass das Platzieren von $N \cdot M$ Blöcken das Gitter vollständig bedeckt.

Deine Aufgabe ist es, eine Strategie zu entwickeln, mit der Julian das Spiel gewinnen kann. Es lässt sich beweisen, dass Julian unter den gegebenen Bedingungen die Blöcke immer so platzieren kann, dass ein Sieg garantiert ist, unabhängig von der Farbe der Blöcke, die er später erhält.

Implementierungsdetails

Implementiere zwei Funktionen:

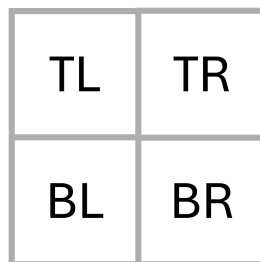
```
void init(int N, int M)
```

- N : die Hälfte der Anzahl an Zeilen im Gitter.

- M : die Hälfte der Anzahl an Spalten im Gitter.
- Die Funktion wird genau einmal pro Testfall aufgerufen, und zwar zu Beginn der Programmausführung.

```
std::pair<int, int> receive_block(int TL, int TR, int BL, int BR)
```

- TL, TR, BL, BR : die Farben der Kacheln oben links, oben rechts, unten links bzw. unten rechts im aktuellen Block, wie in der Abbildung unten dargestellt. Jeder Wert ist entweder 0 (weiß) oder 1 (schwarz).
- Diese Funktion wird nach dem ersten Aufruf von `init` genau $N \cdot M$ Mal pro Testfall aufgerufen.



Diese Funktion sollte ein Paar von ganzen Zahlen (i, j) zurückgeben, wobei i die Zeilenkoordinate und j die Spaltenkoordinate der Zelle ist, in der die obere linke Kachel dieses Blocks platziert werden soll. Sowohl i als auch j müssen gerade sein, und der 2×2 -Bereich, der vom Block abgedeckt wird, darf sich nicht mit einem zuvor platzierten Block überschneiden.

Falls `receive_block` jemals ein Paar zurückgibt, das diese Anforderungen nicht erfüllt, oder falls nach dem Platzieren des Blocks ein 2×2 -Quadrat aus Zellen vollständig mit schwarzen Kacheln bedeckt wird, beendet der Grader dein Programm sofort, und das Urteil für den Testfall lautet: `Output isn't correct`.

Das Verhalten des Graders ist **nicht adaptiv**. Das bedeutet, dass die Reihenfolge der Blöcke, die Manuel an Julian übergibt, vor dem Aufruf von `init` festgelegt ist.

Einschränkungen

Für jeden Block sei S die Anzahl der schwarzen Kacheln unter seinen vier Kacheln. Das heißt, $S = TL + TR + BL + BR$.

- $1 \leq N, M \leq 100$
- $0 \leq S \leq 3$ für jeden Block.

Teilaufgaben

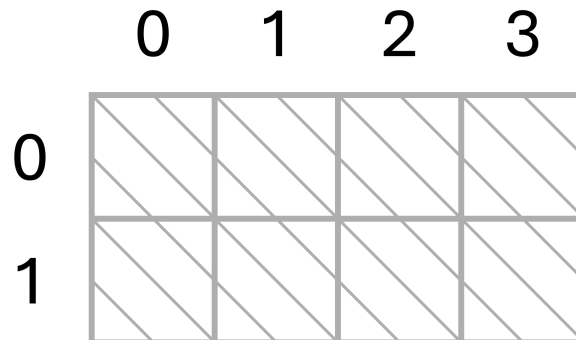
Teilaufgabe	Punktzahl	Zusätzliche Einschränkungen
1	6	$S = 1$ für jeden Block und $N = 2$.
2	16	$S = 3$ für jeden Block. $N = M$, N ist gerade, und jede der vier möglichen Blockfärbungen kommt genau $\frac{N^2}{4}$ Mal vor.
3	10	$S = 1$ für jeden Block.
4	29	$S \leq 2$ für jeden Block.
5	39	Keine weiteren Einschränkungen.

Beispiel

Betrachten wir ein Spiel mit $N = 1$ und $M = 2$, so dass das Gitter 2 Zeilen und 4 Spalten hat. Der Grader ruft zuerst auf:

```
init(1, 2)
```

Zu Beginn sind alle Zellen leer. Das Gitter sieht folgendermaßen aus:



Es müssen $N \cdot M = 2$ Blöcke platziert werden. Angenommen, Manuel gibt einen Block mit drei schwarzen Kacheln und einer weißen Kachel in der rechten oberen Ecke vor. Der Grader ruft auf:

```
receive_block(1, 0, 1, 1)
```

Julian entscheidet sich dafür, diesen Block auf der linken Seite des Gitters zu platzieren, indem er $(0,0)$ zurückgibt.

Das Gitter sieht nun folgendermaßen aus:

	0	1	2	3
0				
1				

Manuel zeigt dann einen weiteren Block mit drei schwarzen Kacheln und einer weißen Kachel in der linken oberen Ecke:

```
receive_block(0, 1, 1, 1)
```

Die einzige verbleibende Zelle mit gerader Zeile und gerader Spalte, die als obere linke Ecke eines 2×2 -Blocks dienen kann, ist $(0,2)$, daher gibt Julian $(0,2)$ zurück. Das Gitter sieht dann folgendermaßen aus:

	0	1	2	3
0				
1				

Kein 2×2 -Quadrat ist vollständig mit schwarzen Kacheln bedeckt, daher hat Julian alle Blöcke erfolgreich platziert, ohne dass Manuel gewinnen konnte. Julian gewinnt das Spiel.

Beispielgrader

Eingabeformat:

```
N M
TL[0] TR[0] BL[0] BR[0]
TL[1] TR[1] BL[1] BR[1]
...
TL[NM-1] TR[NM-1] BL[NM-1] BR[NM-1]
```

Ausgabeformat:

```
R[0] C[0]  
R[1] C[1]  
...  
R[NM-1] C[NM-1]
```

Hierbei sind $R[k]$ und $C[k]$ das Paar von ganzen Zahlen, das vom k -ten Aufruf von `receive_block` zurückgegeben wird.